

COMPLETE COURSE NOTES

Machine Learning.

From first principles to production — how machines find patterns, learn from data, and make predictions.

- I** Foundations of Machine Learning
- II** Preparing the Data — EDA, Cleaning & Preprocessing
- III** Regression
- IV** Classification
- V** Model Tuning & Ensemble Learning
- VI** Unsupervised Learning
- VII** Quick Reference

Where to find what

I

Foundations of Machine Learning

[What is ML?](#) · [AI vs ML vs DL](#) · [Types of ML](#) · [The ML Workflow](#)

II

Preparing the Data

[Exploratory Data Analysis](#) · [Data Cleaning](#) · [Data Preprocessing](#) · [Feature Engineering & Selection](#)

III

Regression

[Linear Regression](#) · [Cost Function & Gradient Descent](#) · [R² & Adjusted R²](#) · [Overfitting vs Underfitting](#) · [Ridge & Lasso](#)

IV

Classification

[Logistic Regression](#) · [Model Evaluation Metrics](#) · [K-Nearest Neighbors](#) · [Naive Bayes](#) · [Decision Trees](#) · [Support Vector Machines](#)

V

Model Tuning & Ensemble Learning

[Cross Validation](#) · [GridSearchCV & RandomizedSearchCV](#) · [Bagging & Random Forest](#) · [Boosting](#) · [Stacking](#)

VI

Unsupervised Learning

[K-Means Clustering](#) · [DBSCAN](#) · [Dimensionality Reduction & PCA](#)

VII

Quick Reference

[Supervised Algorithm Map](#) · [Unsupervised Algorithm Map](#) · [One-Page Cheat Sheet](#)



PART ONE

Foundations of Machine Learning

What machine learning actually is, how it differs from AI and deep learning, the three learning paradigms, and the eleven-step workflow every ML project follows.

1.1 What is Machine Learning?

1.2 AI vs ML vs DL

1.3 Types of Machine Learning

1.4 The ML Project Workflow

1.1

What is Machine Learning?

Machine Learning is a way of teaching computers to learn from data — just like humans learn from experience. Instead of programming every rule by hand, we give machines lots of data and let them discover the patterns on their own.

Traditional Programming

Rules + Data → Result

A human writes every rule by hand.

Machine Learning

Data + Result → Rules (Model)

The machine works out the rules itself.

Real-world examples

YouTube recommendations

Gmail spam detection

Siri / Alexa

Self-driving cars

Face unlock

1.2

AI vs ML vs DL

The three terms are often used interchangeably, but they nest inside one another like Russian dolls.

Layer	Definition	Example
AI	The big umbrella — the science of making machines "smart" like humans	Chess engines, Alexa, self-driving systems
ML	A subset of AI — machines learn from data and improve over time	Recommendation engines, spam filters
DL	A subset of ML — uses neural networks, inspired by the brain, for complex data	Face recognition, ChatGPT, self-driving vision

1.3

Types of Machine Learning

	Supervised	Unsupervised
Data	Input + labels (output known)	Only input data, no labels
Goal	Learn to predict the label	Learn hidden patterns / structure
Tasks	Classification, Regression	Clustering, Dimensionality Reduction, Anomaly Detection

- **Supervised Learning** — like a student taught by a teacher. We give the machine input and the correct answer, and it learns to predict.
- **Unsupervised Learning** — no labels, just input. The model finds hidden structure: groups similar points, reduces dimensions, spots outliers.
- **Reinforcement Learning** — like training a dog. No direct teaching; reward good behaviour, punish or ignore bad. Learns by trial and error.

1.4

The ML Project Workflow

- 2 Data Collection
- 3 Exploratory Data Analysis (EDA)
- 4 Data Preprocessing / Cleaning
- 5 Feature Selection & Engineering
- 6 Split the Dataset (Train / Test)
- 7 Model Selection
- 8 Model Training
- 9 Model Evaluation
- 10 Hyperparameter Tuning
- 11 Model Testing / Validation

II

PART TWO

Preparing the Data

Roughly eighty percent of real ML work — understanding the data, fixing what's broken in it, transforming it into a usable format, and choosing which signals actually matter.

2.1 Exploratory Data Analysis (EDA)

2.2 Data Cleaning

2.3 Data Preprocessing

2.4 Feature Engineering & Selection



2.1

Exploratory Data Analysis (EDA)

EDA is the process of analysing, visualising and understanding data *before* building any model — detective work that reveals what the data is really saying.

Why EDA matters

- You can't clean what you don't understand
- Helps spot mistakes, biases, or limitations in the data
- Guides the direction of cleaning and feature engineering
- Tells the "story" behind the numbers

The EDA checklist

Step	What it reveals
Viewing the data	<code>head()</code> , <code>tail()</code> , <code>shape</code> , <code>info()</code> — columns & data types
Summary statistics	Mean, median, mode, std, min, max, quartiles — spread & central tendency
Value counts	Unique values per column — ideal for categorical columns
Missing value analysis	Where are the gaps, and what percentage is missing
Visualisations	Histograms (distribution), boxplots (outliers/spread), bar plots (categories), correlation heatmaps, scatter plots
Target exploration	How the output relates to the other variables

REMEMBER

EDA tells you what's wrong with the data. Data Cleaning is where you actually fix it.

2.2

Data Cleaning

1. **Handle missing values** — drop rows/columns only if very few are missing; otherwise impute (mean/median for numeric, mode for categorical), or use advanced methods (regression, KNN, interpolation).
2. **Remove duplicates** — detect and drop exact duplicate rows.
3. **Fix data types** — e.g. numbers stored as strings, dates stored as plain text.
4. **Handle inconsistent categories** — unify "Male" / "male" / "MALE" → one format; "Yes" / "yes" / "Y" → one format.
5. **Detect & handle outliers** — spot them with boxplots, IQR or Z-score, then remove (if clearly wrong) or cap (e.g. to the 95th percentile).
6. **Fix logic / domain errors** — e.g. age = -5, BMI = 150 → replace with the mean/median or remove the row.

2.3

Data Preprocessing

Where cleaning fixes *mistakes*, preprocessing turns already-valid data into a format models can actually use.

Encoding categorical variables

Label Encoding (ordinal)

For ordered categories, e.g. Low / Medium / High.

One-Hot Encoding (nominal)

For non-ordered categories, e.g. region.

Feature transformation

Log or square-root transforms fix right- or left-skewed distributions.

Feature scaling

Brings numeric values onto the same scale — essential for distance-based algorithms like KNN and K-Means.

Method	What it does
Normalization (Min-Max)	Scales values into the range [0, 1]
Standardization (Z-score)	Rescales so mean = 0 and standard deviation = 1

2.4

Feature Engineering & Selection

Feature Engineering creates new features or transforms existing ones to expose patterns a model can learn from — models don't know domain logic, so we hand them the right signals directly (mathematical combinations, target-based flags, binning, time-based features).

Feature Selection keeps only the most useful features and drops the rest — it reduces noise and overfitting, speeds up training, improves accuracy, and makes the model easier to interpret.

Method type	How it works
Filter methods	Pure statistics — correlation matrix, Chi-square test (categorical vs categorical), ANOVA F-test (numeric vs categorical target)
Wrapper methods	Iteratively try different feature subsets and keep the best-performing one
Embedded methods	Built into the model itself — Lasso Regression shrinks coefficients to zero; tree models (Random Forest, XGBoost) output feature-importance scores

THE CURSE OF DIMENSIONALITY

Too many features (say, 150) makes it very hard to find real patterns. Distance-based algorithms (KNN, K-Means, DBSCAN) start to fail, and storage/training time balloon. The fix is **Dimensionality Reduction** — either Feature Selection or Feature Extraction (e.g. PCA, covered in Part VI).



Regression

Predicting continuous, numeric outcomes — fitting the best possible line through the data, measuring how good that fit really is, and keeping the model from memorising noise.

-
- 3.1 Linear Regression
 - 3.2 Cost Function & Gradient Descent
 - 3.3 R^2 & Adjusted R^2
 - 3.4 Overfitting vs Underfitting
 - 3.5 Ridge & Lasso Regression

3.1

Linear Regression

Goal: predict a continuous output (e.g. Salary from Age) by fitting a **best-fit line** through the data.

SIMPLE LINEAR REGRESSION

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1$$

θ_0 (or b) is the intercept · θ_1 (or m) is the slope. Training means finding the values of θ_0, θ_1 that minimise error.

MULTIPLE LINEAR REGRESSION

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

3.2

Cost Function & Gradient Descent

The gap between prediction and actual value is the **residual error**. Training minimises the average squared error across every point:

COST FUNCTION $J(\theta)$

$$J(\theta_0, \theta_1) = (1/2m) \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Plotting $J(\theta_1)$ against θ_1 traces a bowl-shaped curve. The bottom of the bowl is the **Global Minima** — the perfect values of θ . **Gradient Descent** is the algorithm that walks down that bowl step by step, and the **Learning Rate (α)** controls how big each step is on the way to convergence.

3.3

R^2 & Adjusted R^2

R-SQUARED

$$R^2 = 1 - (\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2) = 1 - (\text{Sum of Residuals} / \text{Sum of Total})$$

Always between 0 and 1 (e.g. 0.86 → 86% of the variance is explained).

The problem: R^2 always increases as you add more features — even useless ones. **Adjusted R^2** fixes this by penalising unnecessary features:

ADJUSTED R-SQUARED

$$\text{Adj. } R^2 = 1 - [(1-R^2)(n-1) / (n-p-1)]$$

n = number of rows · p = number of features. If Adjusted R^2 drops when a feature is added, that feature isn't actually helping.

Model	R ²	Adj. R ²	Features (p)	Verdict
A	88%	89%	2	Good — extra feature earns its place
B	89%	86%	5	Worse — extra features are just noise

3.4

Overfitting vs Underfitting

	Overfitting	Perfect fit	Underfitting
Training score	Very high	High	Low
Testing score	Poor	High	Low
Bias / Variance	Low bias, high variance	Low bias, low variance	High bias , high variance

Example	Train	Test	Diagnosis
M1	92%	64%	Overfitting
M2	92%	90%	Perfect / generalised model
M3	68%	64%	Underfitting

3.5

Ridge & Lasso Regression

Both add a penalty term to the cost function to keep the model from overfitting:

REGULARISED COST FUNCTION

$$J(\theta) = (1/2m) \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda(\text{slope})^2$$

λ controls how strongly coefficients are shrunk. Lasso can shrink a coefficient all the way to zero — effectively performing feature selection.



Classification

Predicting categories instead of numbers — from probability curves and distance voting to trees, margins and Bayesian reasoning — plus the metrics used to judge whether any of it actually worked.

4.1 Logistic Regression

4.2 Model Evaluation Metrics

4.3 K-Nearest Neighbors

4.4 Naïve Bayes

4.5 Decision Trees

4.6 Support Vector Machines

4.1

Logistic Regression

Used when the output is **categorical** (e.g. Obese: Yes/No) rather than continuous. A straight line has two problems here: outliers pull it around, and its output can go below 0 or above 1 — not a valid probability.

The fix is to squash the line into an S-shaped curve with the **Sigmoid function**:

SIGMOID FUNCTION

$$h_{\theta}(x) = 1 / (1 + e^{-(\theta_0 + \theta_1 x_1)})$$

$e = 2.71828$ (Euler's number). Output is always between 0 and 1, read as a probability — e.g. $> 50\% \rightarrow$ classified Obese.

Because the curve isn't a straight line, squared error doesn't work as a cost function — logistic regression instead uses **Log Loss**:

LOG LOSS (BINARY CROSS-ENTROPY)

$$\text{Log Loss} = -(1/m) \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

Close, correct predictions score very low loss; confident wrong predictions are penalised heavily. Training finds θ at the global minima of this loss curve.

4.2

Model Evaluation Metrics

	Predicted 1	Predicted 0
Actual 1	TP	FN (Type II error)
Actual 0	FP (Type I error)	TN

Worked example: TP = 190, FN = 10, FP = 10, TN = 40 (total = 250)

Metric	Formula	Use it when...
Accuracy	$(TP+TN) / \text{Total} \rightarrow 92\%$	Classes are roughly balanced
Precision	$TP / (TP+FP) \rightarrow 95\%$	False positives are costly (e.g. spam detection)
Recall	$TP / (TP+FN) \rightarrow 95\%$	False negatives are costly (e.g. disease detection)
F1-Score	$2 \times P \times R / (P+R)$	You want a balance of precision and recall

4.3

K-Nearest Neighbors (KNN)

Classifies a new point by the majority class among its K nearest neighbours (Euclidean distance).

- 1 Choose K — a hyperparameter (e.g. $K = 5$). K should be **odd** to avoid tie votes.

- 2 Calculate the distance from the new point to every existing point (Euclidean Distance).
- 3 Sort the distances and take the K nearest points.
- 4 Let the majority class among those K neighbours decide the prediction.

KNN also works for **regression** (average the K neighbours instead of voting) and generalises to any number of dimensions, not just 2D. The best K is usually found by testing several values with Cross Validation and keeping whichever gives the best accuracy.

4.4

Naive Bayes

Built on **Bayes' Theorem**, and "naive" because it assumes every feature is conditionally independent of the others.

Independent events

$P(A \text{ and } B) = P(A) \times P(B)$
e.g. two coin flips

Dependent events

$P(A \text{ and } B) = P(A) \times P(B|A)$
e.g. drawing cards, no replacement

BAYES' THEOREM

$$P(A|B) = [P(A) \times P(B|A)] / P(B)$$

Applied to classification with independent features $x_1 \dots x_4$ and class y :

$$P(y|x_1, x_2, x_3, x_4) \propto P(y) \times P(x_1|y) \times P(x_2|y) \times P(x_3|y) \times P(x_4|y)$$

Compute this for every class and predict whichever comes out highest.

4.5

Decision Trees

A tree of Root, Branches and Leaves that splits data using if/else conditions on features.

- 1 Start with the full dataset.
- 2 Choose the best feature to split on.
- 3 Create branches from that split.
- 4 Repeat the process for every branch.
- 5 Stop when a node is pure, or the max depth is reached.

Entropy & Information Gain — how the best feature is chosen

ENTROPY

$$\text{Entropy}(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

Example: 14 rows, 9 Yes / 5 No $\rightarrow$ Entropy = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$

INFORMATION GAIN

$$IG(S, \text{feature}) = \text{Entropy}(S) - \sum_v (|S_v| / |S|) \cdot \text{Entropy}(S_v)$$

Compute IG for every candidate feature — the one with the highest IG becomes the Root node.

4.6

Support Vector Machine (SVM)

Finds the line or plane (**hyperplane**) that separates two classes with the **maximum margin** between them.

- **Hyperplane** — the separating boundary (a line in 2D, a plane in 3D)
- **Margin** — the gap between the hyperplane and the nearest points of each class
- **Support Vectors** — the closest points that define/"support" the margin

DECISION BOUNDARY

$$wX + b = 0$$

$wX + b > 0 \rightarrow \text{Class 1}$ · $wX + b < 0 \rightarrow \text{Class 2}$

KERNEL TRICK

When classes aren't linearly separable (e.g. one class ringed by the other), a **kernel function** projects the data into a higher dimension where a clean hyperplane finally exists.

PART FIVE

Model Tuning & Ensemble Learning

Finding the best hyperparameters systematically, and combining many models into one that outperforms any of them alone.

5.1 Cross Validation

5.2 GridSearchCV & RandomizedSearchCV

5.3 Ensemble Learning — Bagging, Boosting & Stacking

5.1

Cross Validation

Hyperparameters control model behaviour (tree depth, K in KNN, learning rate...). Picked randomly, they lead to overfitting, underfitting, or just weak performance — the goal of tuning is to find the combination that generalises best.

Before tuning, split the data into K folds. Each fold takes a turn as the test set while the rest train the model, then the accuracies are averaged for a far more reliable estimate than a single train/test split.

EXAMPLE — 5-FOLD CV

82%, 81%, 86%, 85%, (fold 5) → average $\approx$ 83.5%

5.2

GridSearchCV & RandomizedSearchCV

Method	How it searches
Manual search	Try hyperparameter values by hand
GridSearchCV	Exhaustively tries every combination of the specified values, scoring each with cross-validation
RandomizedSearchCV	Randomly samples a fixed number of combinations instead of all of them — much faster on large search spaces

Example: KNN with $N=[3,5,7,9,11,13] \times \text{weights}=[\text{uniform,distance}] \times \text{metric}=[\text{manhattan,euclidean}] = 24$ combinations for GridSearchCV. For something like XGBoost ($\text{learning_rate} \times \text{n_estimators} \times \text{max_depth} \approx 2,000$ combinations), RandomizedSearchCV samples a manageable subset instead of trying them all.

5.3

Ensemble Learning

"We hear the crowd" — combine several models to get a prediction that's more robust than any single one. Train $M_1, M_2, M_3, M_4 \dots$ on the data, then combine their predictions by majority vote (classification) or averaging (regression).

BAGGING

Bootstrap Aggregation

Split the data into random subsets (with replacement), train the *same* type of model on each subset independently and in parallel, then combine by majority vote or averaging. **Random Forest** is Bagging applied to Decision Trees — e.g. 100 trees, each trained on a random subset of rows *and* a random subset of features. Bagging fixes **overfitting** (high variance → low variance, generalised).

BOOSTING

Sequential Correction

Models are trained one after another, each new one trying to fix the errors of the last — many weak learners combined into one strong learner:

$$\hat{f} = \alpha_1(m_1) + \alpha_2(m_2) + \alpha_3(m_3) + \dots + \alpha_n(m_n)$$

Boosting fixes **underfitting** (high bias → low bias, generalised). Common variants: **AdaBoost**, **Gradient Boosting**, **XGBoost**.

STACKING

Meta-Model Combination

Train several different base models (e.g. SVM, Logistic Regression, Decision Tree) on the training data. Their predictions on a held-out slice become the *input features* for a final meta-model (e.g. KNN), which makes the final call.

Example: 1,000 points split 800 train / 200 test. Base models train on the 800; their predictions on the 200 (SVM_pred, LR_pred, DT_pred) become a small new dataset fed into the meta-model.

VI

PART SIX

Unsupervised Learning

No labels, no teacher — just data. Grouping similar points together, and compressing many features down into the few that actually carry the signal.

6.1 K-Means Clustering

6.2 DBSCAN

6.3 Dimensionality Reduction & PCA

6.1

K-Means Clustering

- 1 Decide the number of clusters, K (e.g. K = 4).
- 2 Initialise centroids — random starting cluster centres.
- 3 Assign each point to its nearest centroid (Euclidean Distance).
- 4 Recompute each centroid as the mean of its assigned points.
- 5 Repeat steps 3–4 until the centroids stop moving (convergence).

CHOOSING K — THE ELBOW METHOD

$$WCSS = d_1^2 + d_2^2 + \dots + d_n^2$$

Plot WCSS (Within-Cluster Sum of Squares) against K — the point where the curve bends and flattens ("the elbow") is the best K.

LIMITATIONS OF K-MEANS

1. K must be chosen manually · 2. Outliers pull the centroid off-centre · 3. Struggles with non-circular, irregularly shaped clusters

6.2

DBSCAN

Where K-Means is **centroid-based**, DBSCAN is **density-based** — and it doesn't require you to pre-specify the number of clusters (a non-parametric algorithm).

It draws an **Epsilon radius** around each point; points that are close enough and densely packed form a cluster, while sparse, far-away points become noise/outliers. This makes DBSCAN far more robust than K-Means to irregular shapes and outliers.

6.3

Dimensionality Reduction & PCA

Too many features causes the same **Curse of Dimensionality** introduced in Part II — hard to find patterns, more storage and training time, and broken distance-based algorithms.

Feature Selection

Simply drop the less useful features and keep the rest as-is.

Feature Extraction

Combine features into new, fewer ones — e.g. Engine Width + Engine Height → Engine Size.

Reducing dimensions always loses *some* information — but a well-chosen reduction still preserves enough signal to predict well.

PRINCIPAL COMPONENT ANALYSIS (PCA)

Finds new axes — Principal Components (PC1, PC2...) — that capture the **maximum variance** in the data. PC1 captures the most variance; PC2 the next-most, orthogonal to PC1; and so on. Achieved through **Eigen Decomposition**.

Typical use: reduce 3D → 2D, or 50 features → 10, while retaining as much of the original information as possible.

VII

PART SEVEN

Quick Reference

Every algorithm from these notes, mapped by family and summarised on a single page — for the night before the exam.

7.1 Supervised Algorithm Map

7.2 Unsupervised Algorithm Map

7.3 One-Page Cheat Sheet

7.1

Supervised Algorithm Map

Supervised Machine Learning	
Linear Regression	continuous output
Logistic Regression	binary / categorical output
Decision Tree	splits via Entropy / Information Gain
Naïve Bayes	probability, Bayes' Theorem
SVM	maximum-margin hyperplane
KNN	distance-based voting
Ensemble — Bagging	→ Random Forest
Ensemble — Boosting	→ AdaBoost, Gradient Boost, XGBoost
Ensemble — Stacking	→ meta-model combines base models

7.2

Unsupervised Algorithm Map

Unsupervised Machine Learning	
K-Means	centroid-based · needs K · Elbow Method
DBSCAN	density-based · no K needed · handles outliers
PCA	feature extraction · max variance · eigen decomposition

7.3

One-Page Cheat Sheet

Algorithm	Type	Key idea
Linear Regression	Regression	Best-fit line, minimise squared error
Ridge / Lasso	Regression	Linear regression + penalty term (λ)
Logistic Regression	Classification	Sigmoid curve + log loss, outputs a probability
KNN	Classification / Regression	Majority vote / average of K nearest points
Naive Bayes	Classification	Bayes' Theorem, assumes independent features
Decision Tree	Classification / Regression	If/else splits chosen by Information Gain
SVM	Classification	Max-margin hyperplane, kernel trick for non-linear data
Random Forest	Ensemble (Bagging)	Many trees, random rows & features, majority vote
AdaBoost / XGBoost	Ensemble (Boosting)	Sequential weak learners fixing prior errors
Stacking	Ensemble	Meta-model learns from base-model predictions
K-Means	Unsupervised (Clustering)	Centroid-based grouping, chosen via Elbow Method
DBSCAN	Unsupervised (Clustering)	Density-based, handles noise & irregular shapes
PCA	Unsupervised (Dim. Reduction)	Projects data onto max-variance axes

End of Notes

Foundations · Data Prep · Regression · Classification · Tuning & Ensembles · Unsupervised Learning